Right Sibling Tree

AlgoExpert · Very Hard · Binary Trees

The problem

- Transform in place so every node's right pointer points to the next node on the same level (null at the end of a level); left pointers keep the first child.
- Level-by-level, O(1) extra space using the already-linked level above.

Intuition

- Process top-down: a left child's sibling is the right child, and a right child's sibling is the parent's sibling's left child.

Approach

- 1. Recurse with parent and side
- 2. Save the old right child
- 3. Set right to the sibling
- 4. Recurse on children

Java solution

```java
public class RightSiblingTree {
    public static BinaryTree rightSiblingTree(BinaryTree root) {
        java.util.List<BinaryTree> level = java.util.List.of(root);
        while (!level.isEmpty()) {
            java.util.List<BinaryTree> next = new java.util.ArrayList<>();
            for (BinaryTree n : level) { if (n.left != null) next.add(n.left); if (n.right != null) next.add(n.right); }
            for (int i = 0; i < level.size(); i++) level.get(i).right = i + 1 < level.size() ? level.get(i + 1) : null;
            level = next;
        }
        return root;
    }
}
```

Complexity

- Time: O(n)
- Space: O(1) (iterative, besides an original-children snapshot per level)

Recap

- Process top-down: a left child's sibling is the right child, and a right child's sibling is the parent's sibling's left child.
- Complexity: O(n)